4.11 Data-Definition Language

In most of our discussions of SQL and relational databases, we have accepted a set of relations as given. Of course, the set of relations in a database must be specified to the system by means of a data definition language (DDL).

The SQL DDL allows specification of not only a set of relations, but also information about each relation, including

- The schema for each relation
- The domain of values associated with each attribute
- The integrity constraints
- The set of indices to be maintained for each relation
- The security and authorization information for each relation
- The physical storage structure of each relation on disk

We discuss here schema definition and domain values; we defer discussion of the other SQL DDL features to Chapter 6.

4.11.1 Domain Types in SQL

The SQL standard supports a variety of built-in domain types, including:

- **char**(n): A fixed-length character string with user-specified length n. The full form, **character**, can be used instead.
- **varchar**(n): A variable-length character string with user-specified maximum length n. The full form, **character varying**, is equivalent.
- **int**: An integer (a finite subset of the integers that is machine dependent). The full form, **integer**, is equivalent.
- **smallint**: A small integer (a machine-dependent subset of the integer domain type).
- **numeric**(p, d): A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point. Thus, **numeric**(3,1) allows 44.5 to be stored exactly, but neither 444.5 or 0.32 can be stored exactly in a field of this type.
- **real, double precision**: Floating-point and double-precision floating-point numbers with machine-dependent precision.
- **float**(n): A floating-point number, with precision of at least n digits.
- **date**: A calendar date containing a (four-digit) year, month, and day of the month.

- **time**: The time of day, in hours, minutes, and seconds. A variant, **time**(p), can be used to specify the number of fractional digits for seconds (the default being 0). It is also possible to store time zone information along with the time.
- **timestamp**: A combination of **date** and **time**. A variant, **timestamp**(p), can be used to specify the number of fractional digits for seconds (the default here being 6).

Date and time values can be specified like this:

date '2001-04-25'
time '09:30:00'
timestamp '2001-04-25 10:29:01.45'

Dates must be specified in the format year followed by month followed by day, as shown. The seconds field of **time** or **timestamp** can have a fractional part, as in the timestamp above. We can use an expression of the form **cast** e **as** t to convert a character string (or string valued expression) e to the type t, where t is one of **date, time**, or **timestamp**. The string must be in the appropriate format as illustrated at the beginning of this paragraph.

To extract individual fields of a **date** or **time** value d, we can use **extract** (*field* **from** d), where *field* can be one of **year, month, day, hour, minute**, or **second**.

SQL allows comparison operations on all the domains listed here, and it allows both arithmetic and comparison operations on the various numeric domains. SQL also provides a data type called **interval**, and it allows computations based on dates and times and on intervals. For example, if x and y are of type **date**, then $x - y$ is an interval whose value is the number of days from date x to date y. Similarly, adding or subtracting an interval to a date or time gives back a date or time, respectively.

It is often useful to compare values from **compatible** domains. For example, since every small integer is an integer, a comparison $x < y$, where x is a small integer and y is an integer (or vice versa), makes sense. We make such a comparison by casting small integer x as an integer. A transformation of this sort is called a **type coercion**. Type coercion is used routinely in common programming languages, as well as in database systems.

As an illustration, suppose that the domain of *customer-name* is a character string of length 20, and the domain of *branch-name* is a character string of length 15. Although the string lengths might differ, standard SQL will consider the two domains compatible.

As we discussed in Chapter 3, the *null* value is a member of all domains. For certain attributes, however, null values may be inappropriate. Consider a tuple in the *customer* relation where *customer-name* is null. Such a tuple gives a street and city for an anonymous customer; thus, it does not contain useful information. In cases such as this, we wish to forbid null values, and we do so by restricting the domain of *customer-name* to exclude null values.

SQL allows the domain declaration of an attribute to include the specification **not null** and thus prohibits the insertion of a null value for this attribute. Any database modification that would cause a null to be inserted in a **not null** domain generates

an error diagnostic. There are many situations where we want to avoid null values. In particular, it is essential to prohibit null values in the primary key of a relation schema. Thus, in our bank example, in the *customer* relation, we must prohibit a null value for the attribute *customer-name*, which is the primary key for *customer*.

4.11.2 Schema Definition in SQL

We define an SQL relation by using the **create table** command:

create table $r(A_1D_1, A_2D_2, \ldots, A_nD_n,$
$\langle$integrity-constraint$_1\rangle$,
$\ldots,$
$\langle$integrity-constraint$_k\rangle)$

where *r* is the name of the relation, each A_i is the name of an attribute in the schema of relation *r*, and D_i is the domain type of values in the domain of attribute A_i. The allowed integrity constraints include

- **primary key** $(A_{j_1}, A_{j_2}, \ldots, A_{j_m})$: The **primary key** specification says that attributes $A_{j_1}, A_{j_2}, \ldots, A_{j_m}$ form the primary key for the relation. The primary key attributes are required to be *non-null* and *unique*; that is, no tuple can have a null value for a primary key attribute, and no two tuples in the relation can be equal on all the primary-key attributes.[1] Although the primary key specification is optional, it is generally a good idea to specify a primary key for each relation.
- **check**(*P*): The **check** clause specifies a predicate *P* that must be satisfied by every tuple in the relation.

The **create table** command also includes other integrity constraints, which we shall discuss in Chapter 6.

Figure 4.8 presents a partial SQL DDL definition of our bank database. Note that, as in earlier chapters, we do not attempt to model precisely the real world in the bank-database example. In the real world, multiple people may have the same name, so *customer-name* would not be a primary key *customer*; a *customer-id* would more likely be used as a primary key. We use *customer-name* as a primary key to keep our database schema simple and short.

If a newly inserted or modified tuple in a relation has null values for any primary-key attribute, or if the tuple has the same value on the primary-key attributes as does another tuple in the relation, SQL flags an error and prevents the update. Similarly, it flags an error and prevents the update if the **check** condition on the tuple fails.

By default **null** is a legal value for every attribute in SQL, unless the attribute is specifically stated to be **not null**. An attribute can be declared to be not null in the following way:

account-number **char**(10) **not null**

1. In SQL-89, primary-key attributes were not implicitly declared to be **not null**; an explicit **not null** declaration was required.

```
create table customer
    (customer-name    char(20),
     customer-street  char(30),
     customer-city    char(30),
     primary key (customer-name))

create table branch
    (branch-name      char(15),
     branch-city      char(30),
     assets           integer,
     primary key (branch-name),
     check (assets >= 0))

create table account
    (account-number   char(10),
     branch-name      char(15),
     balance          integer,
     primary key (account-number),
     check (balance >= 0))

create table depositor
    (customer-name    char(20),
     account-number   char(10),
     primary key (customer-name, account-number))
```

Figure 4.8 SQL data definition for part of the bank database.

SQL also supports an integrity constraint

$$\textbf{unique}\ (A_{j_1}, A_{j_2}, \ldots, A_{j_m})$$

The **unique** specification says that attributes $A_{j_1}, A_{j_2}, \ldots, A_{j_m}$ form a candidate key; that is, no two tuples in the relation can be equal on all the primary-key attributes. However, candidate key attributes are permitted to be null unless they have explicitly been declared to be **not null**. Recall that a null value does not equal any other value. The treatment of nulls here is the same as that of the **unique** construct defined in Section 4.6.4.

A common use of the **check** clause is to ensure that attribute values satisfy specified conditions, in effect creating a powerful type system. For instance, the **check** clause in the **create table** command for relation *branch* checks that the value of **assets** is nonnegative. As another example, consider the following:

```
create table student
    (name             char(15)  not null,
     student-id       char(10),
     degree-level     char(15),
     primary key (student-id),
     check (degree-level in ('Bachelors', 'Masters', 'Doctorate')))
```

Here, we use the **check** clause to simulate an enumerated type, by specifying that *degree-level* must be one of 'Bachelors', 'Masters', or 'Doctorate'. We consider more general forms of **check** conditions, as well as a class of constraints called referential integrity constraints, in Chapter 6.

A newly created relation is empty initially. We can use the **insert** command to load data into the relation. Many relational-database products have special bulk loader utilities to load an initial set of tuples into a relation.

To remove a relation from an SQL database, we use the **drop table** command. The **drop table** command deletes all information about the dropped relation from the database. The command

drop table r

is a more drastic action than

delete from r

The latter retains relation r, but deletes all tuples in r. The former deletes not only all tuples of r, but also the schema for r. After r is dropped, no tuples can be inserted into r unless it is re-created with the **create table** command.

We use the **alter table** command to add attributes to an existing relation. All tuples in the relation are assigned *null* as the value for the new attribute. The form of the **alter table** command is

alter table r **add** A D

where r is the name of an existing relation, A is the name of the attribute to be added, and D is the domain of the added attribute. We can drop attributes from a relation by the command

alter table r **drop** A

where r is the name of an existing relation, and A is the name of an attribute of the relation. Many database systems do not support dropping of attributes, although they will allow an entire table to be dropped.

4.12 Embedded SQL

SQL provides a powerful declarative query language. Writing queries in SQL is usually much easier than coding the same queries in a general-purpose programming language. However, a programmer must have access to a database from a general-purpose programming language for at least two reasons:

1. Not all queries can be expressed in SQL, since SQL does not provide the full expressive power of a general-purpose language. That is, there exist queries that can be expressed in a language such as C, Java, or Cobol that cannot be expressed in SQL. To write such queries, we can embed SQL within a more powerful language.

SQL is designed so that queries written in it can be optimized automatically and executed efficiently—and providing the full power of a programming language makes automatic optimization exceedingly difficult.

2. Nondeclarative actions—such as printing a report, interacting with a user, or sending the results of a query to a graphical user interface—cannot be done from within SQL. Applications usually have several components, and querying or updating data is only one component; other components are written in general-purpose programming languages. For an integrated application, the programs written in the programming language must be able to access the database.

The SQL standard defines embeddings of SQL in a variety of programming languages, such as C, Cobol, Pascal, Java, PL/I, and Fortran. A language in which SQL queries are embedded is referred to as a *host* language, and the SQL structures permitted in the host language constitute *embedded* SQL.

Programs written in the host language can use the embedded SQL syntax to access and update data stored in a database. This embedded form of SQL extends the programmer's ability to manipulate the database even further. In embedded SQL, all query processing is performed by the database system, which then makes the result of the query available to the program one tuple (record) at a time.

An embedded SQL program must be processed by a special preprocessor prior to compilation. The preprocessor replaces embedded SQL requests with host-language declarations and procedure calls that allow run-time execution of the database accesses. Then, the resulting program is compiled by the host-language compiler. To identify embedded SQL requests to the preprocessor, we use the EXEC SQL statement; it has the form

EXEC SQL <embedded SQL statement > END-EXEC

The exact syntax for embedded SQL requests depends on the language in which SQL is embedded. For instance, a semicolon is used instead of END-EXEC when SQL is embedded in C. The Java embedding of SQL (called SQLJ) uses the syntax

SQL { <embedded SQL statement > };

We place the statement SQL INCLUDE in the program to identify the place where the preprocessor should insert the special variables used for communication between the program and the database system. Variables of the host language can be used within embedded SQL statements, but they must be preceded by a colon (:) to distinguish them from SQL variables.

Embedded SQL statements are similar in form to the SQL statements that we described in this chapter. There are, however, several important differences, as we note here.

To write a relational query, we use the **declare cursor** statement. The result of the query is not yet computed. Rather, the program must use the **open** and **fetch** commands (discussed later in this section) to obtain the result tuples.

Consider the banking schema that we have used in this chapter. Assume that we have a host-language variable *amount*, and that we wish to find the names and cities of residence of customers who have more than *amount* dollars in any account. We can write this query as follows:

```
EXEC SQL
        declare c cursor for
        select customer-name, customer-city
        from depositor, customer, account
        where depositor.customer-name = customer.customer-name and
                account.account-number = depositor.account-number and
                account.balance > :amount
END-EXEC
```

The variable *c* in the preceding expression is called a *cursor* for the query. We use this variable to identify the query in the **open** statement, which causes the query to be evaluated, and in the **fetch** statement, which causes the values of one tuple to be placed in host-language variables.

The **open** statement for our sample query is as follows:

```
EXEC SQL open c END-EXEC
```

This statement causes the database system to execute the query and to save the results within a temporary relation. The query has a host-language variable (*:amount*); the query uses the value of the variable at the time the **open** statement was executed.

If the SQL query results in an error, the database system stores an error diagnostic in the SQL communication-area (SQLCA) variables, whose declarations are inserted by the SQL INCLUDE statement.

An embedded SQL program executes a series of **fetch** statements to retrieve tuples of the result. The **fetch** statement requires one host-language variable for each attribute of the result relation. For our example query, we need one variable to hold the *customer-name* value and another to hold the *customer-city* value. Suppose that those variables are *cn* and *cc*, respectively. Then the statement:

```
EXEC SQL fetch c into :cn, :cc END-EXEC
```

produces a tuple of the result relation. The program can then manipulate the variables *cn* and *cc* by using the features of the host programming language.

A single **fetch** request returns only one tuple. To obtain all tuples of the result, the program must contain a loop to iterate over all tuples. Embedded SQL assists the programmer in managing this iteration. Although a relation is conceptually a set, the tuples of the result of a query are in some fixed physical order. When the program executes an **open** statement on a cursor, the cursor is set to point to the first tuple of the result. Each time it executes a **fetch** statement, the cursor is updated to point to the next tuple of the result. When no further tuples remain to be processed, the variable SQLSTATE in the SQLCA is set to '02000' (meaning "no data"). Thus, we can use a **while** loop (or equivalent loop) to process each tuple of the result.

We must use the **close** statement to tell the database system to delete the temporary relation that held the result of the query. For our example, this statement takes the form

EXEC SQL **close** *c* END-EXEC

SQLJ, the Java embedding of SQL, provides a variation of the above scheme, where Java iterators are used in place of cursors. SQLJ associates the results of a query with an iterator, and the next() method of the Java iterator interface can be used to step through the result tuples, just as the preceding examples use **fetch** on the cursor.

Embedded SQL expressions for database modification (**update**, **insert**, and **delete**) do not return a result. Thus, they are somewhat simpler to express. A database-modification request takes the form

EXEC SQL < any valid **update, insert,** or **delete**> END-EXEC

Host-language variables, preceded by a colon, may appear in the SQL database-modification expression. If an error condition arises in the execution of the statement, a diagnostic is set in the SQLCA.

Database relations can also be updated through cursors. For example, if we want to add 100 to the *balance* attribute of every *account* where the branch name is "Perryridge", we could declare a cursor as follows.

```
declare c cursor for
select *
from account
where branch-name = 'Perryridge'
for update
```

We then iterate through the tuples by performing **fetch** operations on the cursor (as illustrated earlier), and after fetching each tuple we execute the following code

```
update account
set balance = balance + 100
where current of c
```

Embedded SQL allows a host-language program to access the database, but it provides no assistance in presenting results to the user or in generating reports. Most commercial database products include tools to assist application programmers in creating user interfaces and formatted reports. We discuss such tools in Chapter 5 (Section 5.3).

4.13 Dynamic SQL

The *dynamic SQL* component of SQL allows programs to construct and submit SQL queries at run time. In contrast, embedded SQL statements must be completely present at compile time; they are compiled by the embedded SQL preprocessor. Using dynamic SQL, programs can create SQL queries as strings at run time (perhaps based on

input from the user) and can either have them executed immediately or have them *prepared* for subsequent use. Preparing a dynamic SQL statement compiles it, and subsequent uses of the prepared statement use the compiled version.

SQL defines standards for embedding dynamic SQL calls in a host language, such as C, as in the following example.

```
char * sqlprog = "update account set balance = balance *1.05
        where account-number = ?"
EXEC SQL prepare dynprog from :sqlprog;
char account[10] = "A-101";
EXEC SQL execute dynprog using :account;
```

The dynamic SQL program contains a ?, which is a place holder for a value that is provided when the SQL program is executed.

However, the syntax above requires extensions to the language or a preprocessor for the extended language. An alternative that is very widely used is to use an application program interface to send SQL queries or updates to a database system, and not make any changes in the programming language itself.

In the rest of this section, we look at two standards for connecting to an SQL database and performing queries and updates. One, ODBC, is an application program interface for the C language, while the other, JDBC, is an application program interface for the Java language.

To understand these standards, we need to understand the concept of SQL sessions. The user or application *connects* to an SQL server, establishing a session; executes a series of statements; and finally *disconnects* the session. Thus, all activities of the user or application are in the context of an SQL session. In addition to the normal SQL commands, a session can also contain commands to *commit* the work carried out in the session, or to *rollback* the work carried out in the session.

4.13.1 ODBC**

The **Open DataBase Connectivity** (ODBC) standard defines a way for an application program to communicate with a database server. ODBC defines an **application program interface (API)** that applications can use to open a connection with a database, send queries and updates, and get back results. Applications such as graphical user interfaces, statistics packages, and spreadsheets can make use of the same ODBC API to connect to any database server that supports ODBC.

Each database system supporting ODBC provides a library that must be linked with the client program. When the client program makes an ODBC API call, the code in the library communicates with the server to carry out the requested action, and fetch results.

Figure 4.9 shows an example of C code using the ODBC API. The first step in using ODBC to communicate with a server is to set up a connection with the server. To do so, the program first allocates an SQL environment, then a database connection handle. ODBC defines the types HENV, HDBC, and RETCODE. The program then opens the database connection by using SQLConnect. This call takes several parameters, in-

```
int ODBCexample()
{
    RETCODE error;
    HENV env; /* environment */
    HDBC conn; /* database connection */

    SQLAllocEnv(&env);
    SQLAllocConnect(env, &conn);
    SQLConnect(conn, "aura.bell-labs.com", SQL_NTS, "avi", SQL_NTS,
                "avipasswd", SQL_NTS);
    {
        char branchname[80];
        float balance;
        int lenOut1, lenOut2;
        HSTMT stmt;

        SQLAllocStmt(conn, &stmt);
        char * sqlquery = "select branch_name, sum (balance)
                           from account
                           group by branch_name";
        error = SQLExecDirect(stmt, sqlquery, SQL_NTS);
        if (error == SQL_SUCCESS) {
            SQLBindCol(stmt, 1, SQL_C_CHAR, branchname , 80, &lenOut1);
            SQLBindCol(stmt, 2, SQL_C_FLOAT, &balance, 0 , &lenOut2);
            while (SQLFetch(stmt) >= SQL_SUCCESS) {
                printf (" %s %g\n", branchname, balance);
            }
        }
    }
    SQLFreeStmt(stmt, SQL_DROP);
    SQLDisconnect(conn);
    SQLFreeConnect(conn);
    SQLFreeEnv(env);
}
```

Figure 4.9 ODBC code example.

cluding the connection handle, the server to which to connect, the user identifier, and the password for the database. The constant SQL_NTS denotes that the previous argument is a null-terminated string.

Once the connection is set up, the program can send SQL commands to the database by using SQLExecDirect C language variables can be bound to attributes of the query result, so that when a result tuple is fetched using SQLFetch, its attribute values are stored in corresponding C variables. The SQLBindCol function does this task; the second argument identifies the position of the attribute in the query result, and the third argument indicates the type conversion required from SQL to C. The next argument

gives the address of the variable. For variable-length types like character arrays, the last two arguments give the maximum length of the variable and a location where the actual length is to be stored when a tuple is fetched. A negative value returned for the length field indicates that the value is **null**.

The SQLFetch statement is in a **while** loop that gets executed until SQLFetch returns a value other than SQL_SUCCESS. On each fetch, the program stores the values in C variables as specified by the calls on SQLBindCol and prints out these values.

At the end of the session, the program frees the statement handle, disconnects from the database, and frees up the connection and SQL environment handles. Good programming style requires that the result of every function call must be checked to make sure there are no errors; we have omitted most of these checks for brevity.

It is possible to create an SQL statement with parameters; for example, consider the statement insert into account values(?,?,?). The question marks are placeholders for values which will be supplied later. The above statement can be "prepared," that is, compiled at the database, and repeatedly executed by providing actual values for the placeholders—in this case, by providing an account number, branch name, and balance for the relation *account*.

ODBC defines functions for a variety of tasks, such as finding all the relations in the database and finding the names and types of columns of a query result or a relation in the database.

By default, each SQL statement is treated as a separate transaction that is committed automatically. The call SQLSetConnectOption(conn, SQL_AUTOCOMMIT, 0) turns off automatic commit on connection conn, and transactions must then be committed explicitly by SQLTransact(conn, SQL_COMMIT) or rolled back by SQLTransact(conn, SQL_ROLLBACK).

The more recent versions of the ODBC standard add new functionality. Each version defines *conformance levels*, which specify subsets of the functionality defined by the standard. An ODBC implementation may provide only core level features, or it may provide more advanced (level 1 or level 2) features. Level 1 requires support for fetching information about the catalog, such as information about what relations are present and the types of their attributes. Level 2 requires further features, such as ability to send and retrieve arrays of parameter values and to retrieve more detailed catalog information.

The more recent SQL standards (SQL-92 and SQL:1999) define a **call level interface (CLI)** that is similar to the ODBC interface, but with some minor differences.

4.13.2 JDBC**

The **JDBC** standard defines an API that Java programs can use to connect to database servers. (The word JDBC was originally an abbreviation for "Java Database Connectivity", but the full form is no longer used.) Figure 4.10 shows an example Java program that uses the JDBC interface. The program must first open a connection to a database, and can then execute SQL statements, but before opening a connection, it loads the appropriate drivers for the database by using Class.forName. The first parameter to the getConnection call specifies the machine name where the server

```
public static void JDBCexample(String dbid, String userid, String passwd)
{
    try
    {
        Class.forName ("oracle.jdbc.driver.OracleDriver");
        Connection conn = DriverManager.getConnection(
                "jdbc:oracle:thin:@aura.bell-labs.com:2000:bankdb",
                userid, passwd);
        Statement stmt = conn.createStatement();
        try {
            stmt.executeUpdate(
                "insert into account values('A-9732', 'Perryridge', 1200)");
        } catch (SQLException sqle)
        {
            System.out.println("Could not insert tuple. " + sqle);
        }
        ResultSet rset = stmt.executeQuery(
                "select branch_name, avg (balance)
                from account
                group by branch_name");
        while (rset.next()) {
            System.out.println(rset.getString("branch_name") + " " +
                    rset.getFloat(2));
        }
        stmt.close();
        conn.close();
    }
    catch (SQLException sqle)
    {
        System.out.println("SQLException : " + sqle);
    }
}
```

Figure 4.10 An example of JDBC code.

runs (in our example, aura.bell-labs.com), the port number it uses for communication (in our example, 2000). The parameter also specifies which schema on the server is to be used (in our example, bankdb), since a database server may support multiple schemas. The first parameter also specifies the protocol to be used to communicate with the database (in our example, jdbc:oracle:thin:). Note that JDBC specifies only the API, not the communication protocol. A JDBC driver may support multiple protocols, and we must specify one supported by both the database and the driver. The other two arguments to getConnection are a user identifier and a password.

The program then creates a statement handle on the connection and uses it to execute an SQL statement and get back results. In our example, stmt.executeUpdate executes an update statement. The try { . . . } catch { . . . } construct permits us to

```
PreparedStatement pStmt = conn.prepareStatement(
                    "insert into account values(?,?,?)");
pStmt.setString(1, "A-9732");
pStmt.setString(2, "Perryridge");
pStmt.setInt(3, 1200);
pStmt.executeUpdate();
pStmt.setString(1, "A-9733");
pStmt.executeUpdate();
```

Figure 4.11 Prepared statements in JDBC code.

catch any exceptions (error conditions) that arise when JDBC calls are made, and print an appropriate message to the user.

The program can execute a query by using stmt.executeQuery. It can retrieve the set of rows in the result into a ResultSet and fetch them one tuple at a time using the next() function on the result set. Figure 4.10 shows two ways of retrieving the values of attributes in a tuple: using the name of the attribute (*branch-name*) and using the position of the attribute (2, to denote the second attribute).

We can also create a prepared statement in which some values are replaced by "?", thereby specifying that actual values will be provided later. We can then provide the values by using setString(). The database can compile the query when it is prepared, and each time it is executed (with new values), the database can reuse the previously compiled form of the query. The code fragment in Figure 4.11 shows how prepared statements can be used.

JDBC provides a number of other features, such as **updatable result sets**. It can create an updatable result set from a query that performs a selection and/or a projection on a database relation. An update to a tuple in the result set then results in an update to the corresponding tuple of the database relation. JDBC also provides an API to examine database schemas and to find the types of attributes of a result set.

For more information about JDBC, refer to the bibliographic information at the end of the chapter.

4.14 Other SQL Features **

The SQL language has grown over the past two decades from a simple language with a few features to a rather complex language with features to satisfy many different types of users. We covered the basics of SQL earlier in this chapter. In this section we introduce the reader to some of the more complex features of SQL.

4.14.1 Schemas, Catalogs, and Environments

To understand the motivation for schemas and catalogs, consider how files are named in a file system. Early file systems were flat; that is, all files were stored in a single directory. Current generation file systems of course have a directory structure, with

files stored within subdirectories. To name a file uniquely, we must specify the full path name of the file, for example, /users/avi/db-book/chapter4.tex.

Like early file systems, early database systems also had a single name space for all relations. Users had to coordinate to make sure they did not try to use the same name for different relations. Contemporary database systems provide a three-level hierarchy for naming relations. The top level of the hierarchy consists of **catalogs**, each of which can contain **schemas**. SQL objects such as relations and views are contained within a **schema**.

In order to perform any actions on a database, a user (or a program) must first *connect* to the database. The user must provide the user name and usually, a secret password for verifying the identity of the user, as we saw in the ODBC and JDBC examples in Sections 4.13.1 and 4.13.2. Each user has a default catalog and schema, and the combination is unique to the user. When a user connects to a database system, the default catalog and schema are set up for for the connection; this corresponds to the current directory being set to the user's home directory when the user logs into an operating system.

To identify a relation uniquely, a three-part name must be used, for example,

catalog5.bank-schema.account

We may omit the catalog component, in which case the catalog part of the name is considered to be the default catalog for the connection. Thus if catalog5 is the default catalog, we can use bank-schema.account to identify the same relation uniquely. Further, we may also omit the schema name, and the schema part of the name is again considered to be the default schema for the connection. Thus we can use just account if the default catalog is catalog5 and the default schema is bank-schema.

With multiple catalogs and schemas available, different applications and different users can work independently without worrying about name clashes. Moreover, multiple versions of an application—one a production version, other test versions—can run on the same database system.

The default catalog and schema are part of an **SQL environment** that is set up for each connection. The environment additionally contains the user identifier (also referred to as the *authorization identifier*). All the usual SQL statements, including the DDL and DML statements, operate in the context of a schema. We can create and drop schemas by means of **create schema** and **drop schema** statements. Creation and dropping of catalogs is implementation dependent and not part of the SQL standard.

4.14.2 Procedural Extensions and Stored Procedures

SQL provides a **module** language, which allows procedures to be defined in SQL. A module typically contains multiple SQL procedures. Each procedure has a name, optional arguments, and an SQL statement. An extension of the SQL-92 standard language also permits procedural constructs, such as **for**, **while**, and **if-then-else**, and compound SQL statements (multiple SQL statements between a **begin** and an **end**).

We can store procedures in the database and then execute them by using the **call** statement. Such procedures are also called **stored procedures**. Stored procedures